

P2539R3

Should the output of `std::print` to a terminal be synchronized with the underlying stream?

Published Proposal, 2022-09-27

Author:

[Victor Zverovich](#)

Audience:

LWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

1 Introduction

2 Revision history

3 LEWG Poll (R1)

4 Motivating examples

5 Proposal

6 Wording

References

Informative References

§ 1. Introduction

To prevent mojibake `std::print` may use a native Unicode API when writing to a terminal bypassing the stream buffer. During the review of [P2093] "Formatted output" Tim Song suggested that synchronizing `std::print` with the underlying stream may be beneficial for gradual adoption. This paper presents motivating examples, observes that this problem doesn't normally happen in practice and proposes a minor update to the wording to provide a synchronization guarantee.

§ 2. Revision history

Changes since R2:

- Replaced "If the native Unicode API is used and `stdout` referring to a terminal is buffered by default on the current system, the function flushes the `stream`'s buffer before writing `out`." with "If the native Unicode API is used, the function flushes the `stream`'s buffer before writing `out`." to provide a stronger guarantee per LEWG feedback.
- Replaced "If `stdout` referring to a terminal is buffered by default on the current system, the function flushes the `os`'s buffer before writing `out`." with "If the native Unicode API is used, the function flushes the `os`'s buffer before writing `out`." to provide a stronger guarantee per LEWG feedback.

Changes since R1:

- Added LEWG poll results.
- Replaced "the terminal output is buffered by default" with a more specific "`stdout` referring to a terminal is buffered by default on the current system".

Changes since R0:

- Added another motivating example.
- Split Discussion into multiple sections.
- Added the wording.

§ 3. LEWG Poll (R1)

Poll: Send P2539R1 Should The Output Of `print` To A Terminal Be Synchronized With The Underlying Stream? to Library Working Group for C++23, classified as an addition (P0592R4 bucket

3 item).

SF	F	N	A	SA
11	10	2	1	0

Outcome: Consensus in favor

§ 4. Motivating examples

Consider the following example:

```
printf("first\n");  
std::print("second\n");
```

This will produce the expected output:

```
first  
second
```

because `stdout` is at least line buffered by default.

However, in theory this may reorder the output:

```
printf("first");  
std::print("second");
```

because of buffering in `printf` but not `std::print`. Testing on Windows 10 with MSVC 19.28 and `{fmt}`'s implementation of `print` ([\[FMT\]](#)) showed that the order is preserved in this case as well. This suggests that `stdout` is completely unbuffered by default on this system. This is also confirmed in [\[MS-CRT\]](#):

The `stdout` and `stderr` functions are flushed whenever they are full or, if you are writing to a character device, after each library call.

On other systems the order is preserved too because the output goes through the stream buffer in both cases.

Consider, another example that involves iostreams:

```
struct A {
    int a;
    int b;

    friend std::ostream& operator<<(std::ostream& os, const A& a) {
        std::print(os, "{{a={}, b={}}}", a.a, a.b);
        return os;
    }
};

int main() {
    A a = {2, 4};
    std::cout << "A is " << a << '\n';
}
```

We updated the implementation of `print` for `ostream` in `{fmt}` to use the native Unicode API and verified that there is no reordering in this example either on the same test platform.

§ 5. Proposal

Although the issue appears to be mostly theoretical, it might still be beneficial to clarify in the standard that synchronization is desired. It is possible to guarantee the desired output ordering by flushing the buffer before writing to a terminal in `std::print`. This will incur additional cost but only for the terminal case and when transcoding is needed. Platforms that don't buffer the output like the one we tested should be able to avoid a call to flush.

Neither `{fmt}` ([FMT]) nor Rust ([RUST-STDIO]) do any attempt to provide such synchronization in their implementations of `print`. However, in practice this synchronization appears to be a noop on tested platforms.

§ 6. Wording

Modify subsection "Print functions [print.fun]":

```
void vprint_unicode(FILE* stream, string_view fmt, format_args args);
```

...

Effects: The function initializes an automatic variable via

```
string out = vformat(fmt, args);
```

If `stream` refers to a terminal capable of displaying Unicode, writes `out` to the terminal using the native Unicode API; if `out` contains invalid code units, the behavior is undefined and implementations are encouraged to diagnose it. Otherwise writes `out` to `stream` unchanged.

If the native Unicode API is used, the function flushes the `stream`'s buffer before writing `out`.

Modify subsection "Print [ostream.formatted.print]":

```
void vprint_unicode(ostream& os, string_view fmt, format_args args);  
void vprint_nonunicode(ostream& os, string_view fmt, format_args args);
```

Effects: Behaves as a formatted output function ([ostream.formatted.reqmts]) of `os`, except that:

- failure to generate output is reported as specified below, and
- any exception thrown by the call to `vformat` is propagated without regard to the value of `os.exceptions()` and without turning on `ios_base::badbit` in the error state of `os`.

After constructing a `sentry` object, the function initializes an automatic variable via

```
string out = vformat(os.getloc(), fmt, args);
```

If the function is `vprint_unicode` and `os` is a stream that refers to a terminal capable of displaying Unicode which is determined in an implementation-defined manner, writes `out` to the terminal using the native Unicode API; if `out` contains invalid code units, the behavior is undefined and implementations are encouraged to diagnose it. If the native Unicode API is used, the function flushes the `os`'s buffer before writing `out`.

Otherwise (if `os` is not such a stream or the function is `vprint_nonunicode`), inserts the character sequence `[out.begin(), out.end())` into `os`.

If writing to the terminal or inserting into `os` fails, calls `os.setstate(ios_base::badbit)` (which may throw `ios_base::failure`).

§ References

§ Informative References

[FMT]

Victor Zverovich; et al. [The fmt library](https://github.com/fmtlib/fmt). URL: <https://github.com/fmtlib/fmt>

[MS-CRT]

[C runtime library \(CRT\) reference, Stream I/O](https://docs.microsoft.com/en-us/cpp/c-runtime-library/stream-i-o). URL: <https://docs.microsoft.com/en-us/cpp/c-runtime-library/stream-i-o>

[P2093]

Victor Zverovich. [Formatted output](https://wg21.link/p2093). URL: <https://wg21.link/p2093>

[RUST-STDIO]

[The Rust Programming Language repository, windows_stdio](https://github.com/rust-lang/rust/blob/db492ecd5ba6bd82205612cebb9034710653f0c2/library/std/src/sys/windows/stdio.rs). URL: <https://github.com/rust-lang/rust/blob/db492ecd5ba6bd82205612cebb9034710653f0c2/library/std/src/sys/windows/stdio.rs>